

SciWalker: Synthesizing Scientific Coding Problems with Operator Graphs and Execution Feedback

Chenxi Li^{1,2} Wenxuan Zeng^{2,3} Yun Luo^{2†,‡} Fangchen Yu²
Peng Ye² Yu Cheng⁴ Jun Zhang^{1‡}

¹ The Hong Kong University of Science and Technology ² Shanghai AI Laboratory

³ Tsinghua University ⁴ Nanyang Technological University

[†]Project Lead, [‡]Corresponding authors

ABSTRACT

Improving the scientific coding capabilities of large language models (LLMs) requires high-quality training data. However, such data remain scarce because manually authoring realistic problems is costly and time-consuming, while systematically covering diverse scientific domains and algorithmic combinations remains challenging. To address this, we introduce SciWalker, a framework for synthesizing scientific coding problems through operator-chain sampling and execution feedback. The framework combines scientific library interfaces with operation modes to instantiate operators, organizes them into operator graphs, and samples operator chains as computational workflow cues. Guided by these cues, we adopt LLMs to generate scientifically grounded problem statements, reference solutions, and tests, with failed generations iteratively repaired using execution feedback. By combining structured workflow composition with verification and quality review, SciWalker enables scalable task generation while promoting scientific grounding, computational diversity, and executability. Using this framework, we construct 8,178 high-quality problems spanning 5 scientific domains and 32 subdomains. To evaluate their training utility, we conduct reinforcement learning on Qwen3.5-9B using the GSPO algorithm. This training improves SciCode subproblem accuracy by 9.9 percentage points, from 29.3% to 39.2%, with gains across scientific code generation, code repair, and reasoning benchmarks. The code for SciWalker is available at <https://github.com/lichenx1/SciWalker>.

1 Introduction

Scientific code plays a crucial role in translating scientific theories into computational practice across a wide range of disciplines (Virtanen et al., 2020). High-quality, diverse training data have been shown to improve the scientific reasoning and code generation capabilities of large language models (LLMs) (Zhang et al., 2024; Wei et al., 2024). However, such data remain scarce because constructing realistic scientific coding problems typically requires substantial expert effort to formulate scientific contexts, compose computational procedures, implement reference solutions, and design reliable tests (Tian et al., 2024). Manually carrying out these steps is costly and time-consuming (Lai et al., 2023), while systematically covering diverse scientific domains, subdomains, and algorithmic combinations remains challenging.

To address these challenges, we introduce SciWalker, a framework for scalable synthesis of multistep scientific coding problems. Our central idea is to leverage scientific computing libraries as computational building blocks and compose their interfaces into diverse scientific workflows as illustrated in Figure 1. Specifically, SciWalker combines library interfaces with operation modes to instantiate operators, organizes them into operator graphs, and samples operator chains as workflow cues. Guided by these cues and corresponding scientific contexts, LLMs generate problem statements, reference solutions, and tests with well-defined computational objectives. Execution feedback is then used to identify and iteratively repair failures in the generated code and tests, while quality review further filters the resulting problems.

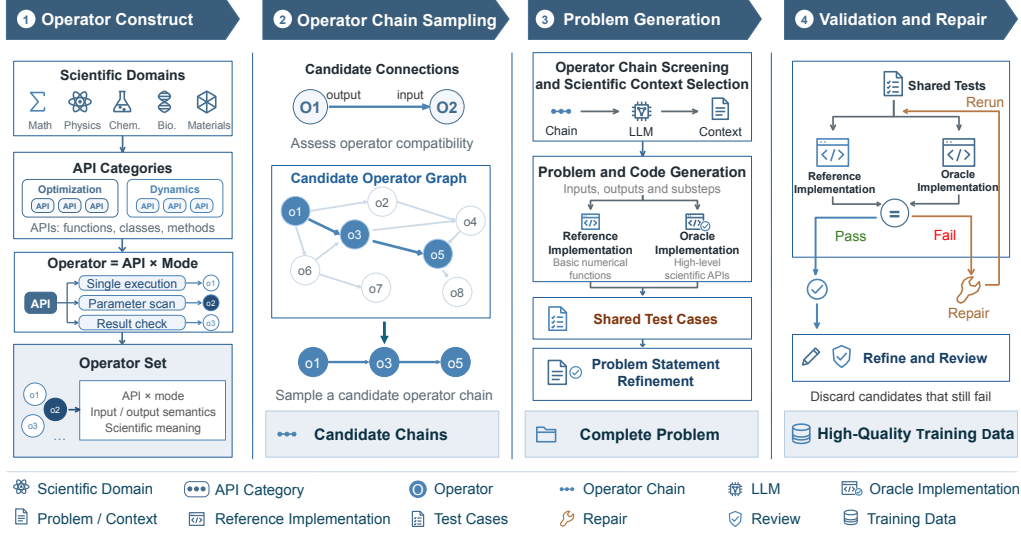


Figure 1: Scientific code training data generation workflow. Scientific library APIs are combined with operation modes to form operators, which are used to construct a graph and sample candidate operator chains. Large language models screen these chains and select scientific contexts, generate tasks, reference implementations, and tests, and then refine the problem statements accordingly. Execution validation, feedback-driven repair, and quality review yield high-quality training data.

Based on SciWalker, we construct a large-scale dataset of 8,178 scientific coding problems spanning five major domains, including mathematics, physics, chemistry, biology, and materials science. The problems are generated from operator chains of varying lengths and subsequently refined through execution-guided repair and quality review, resulting in diverse computational workflows grounded in realistic scientific contexts. Collectively, the dataset contains 37,820 computational substeps, with each problem integrating multiple interdependent operations rather than isolated function calls.

To assess the training value of the generated data, we conduct reinforcement learning on Qwen3.5-9B (Qwen Team, 2026) using step-level execution rewards and GSPO policy updates (Zheng et al., 2025). This training improves SciCode subproblem accuracy from 29.3% to 39.2%, a gain of 9.9 percentage points over the baseline. Gains also extend to scientific code generation, code repair, and reasoning benchmarks, with improvements of 17.3 percentage points on DS-1000 and 6.0 percentage points on MATH-500.

The main contributions of our work are as follows:

- We propose SciWalker, which combines scientific operator composition with execution feedback to guide large language models in automatically generating and revising multistep scientific coding problems.
- We construct 8,178 high-quality scientific problems covering 5 scientific domains and 32 subdomains, accompanied by reference solutions and tests.
- We conduct reinforcement learning experiments to assess the training value of the generated data. The trained model shows improvements on both in-domain scientific coding tasks and out-of-domain code repair and reasoning benchmarks.

2 Related Work

Scientific and code data construction. SciCode (Tian et al., 2024) provides real research problems, reference solutions, and tests curated by scientists to evaluate multistep scientific coding capabilities. Automated data construction approaches draw on existing problems, scientific literature, and code resources. SciInstruct (Zhang et al., 2024) augments existing scientific problems with model-generated reasoning, refined through self-review and revision. WildSci (Liu et al., 2026) synthesizes scientific

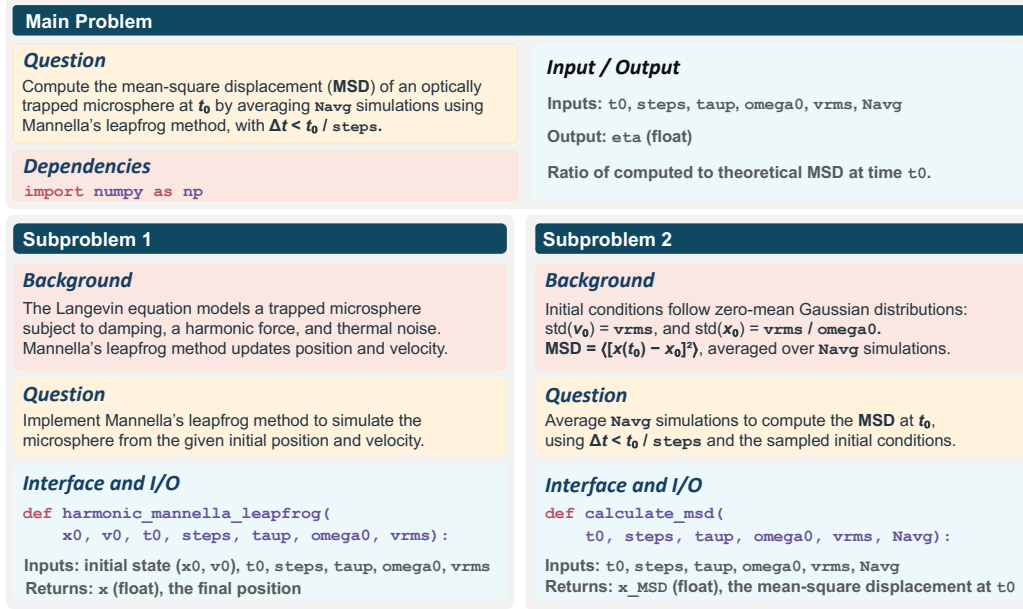


Figure 2: Multi-turn scientific coding problem format.

multiple-choice questions from peer-reviewed literature, supporting reinforcement learning through unambiguous answer evaluation. UniScientist (Li et al., 2026) combines validated scientific claims with evidence retrieval to generate open-ended research questions, accompanied by rubrics revised and validated by language models and domain experts. Magicoder (Wei et al., 2024) uses open-source code snippets to guide the generation of programming tasks. OpenCodeInterpreter (Zheng et al., 2024) trains code models on multi-turn interactions incorporating execution and human feedback for iterative refinement. Scaling multistep scientific coding data requires diverse combinations of scientific computations, coherent tasks with clear scientific objectives, and reference solutions with tests. Our work addresses these requirements through scientific operator composition and scientific context design, using execution feedback to repair and select the generated problems.

3 Synthesizing Scientific Coding Problems with Operator Graphs and Execution Feedback

Scientific coding tasks often involve interdependent computations, where later steps build on functions implemented earlier. We therefore adopt a multi-turn format that preserves these dependencies and supports stepwise validation. Following SciCode (Tian et al., 2024), each problem consists of a main problem and an ordered sequence of subproblems, as illustrated in Figure 2. The main problem defines the overall objective and dependencies. Each subproblem provides a task description, background, and interface specification. At each turn, the model implements the current subproblem using its specification and previously generated code.

Our framework combines LLMs with programmatic workflows to construct scientific problems containing problem statements, reference solutions, and tests. The overall workflow consists of four stages as illustrated in Figure 1: (1) Scientific operator construction; (2) Operator graph sampling; (3) Scientific problem generation; (4) Validation, Repair, and Final refinement.

3.1 Operator Construction

We first organize scientific library interfaces by computational purpose into categories such as optimization, dynamics, and matrix decomposition. Each API, defined as a function, class, or method, is combined with operation modes such as single-case execution, parameter sweeps, and result checking. We define each API $\times$ mode combination as an operator, which serves as a node for graph construction and operator chain sampling. Representing different uses of the same API as distinct

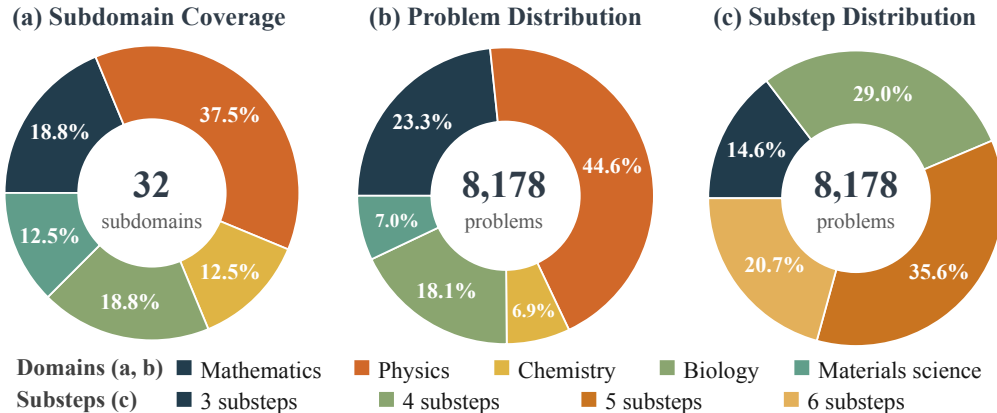


Figure 3: Dataset coverage and composition. (a) Domain distribution of 32 subdomains. (b) Domain distribution of 8,178 high-quality problems. (c) Distribution of substep counts across the same 8,178 problems. Sector areas represent category proportions, with percentages labeled on the sectors and categories identified in the legends.

operators expands a finite collection of APIs into a larger operator space, enabling more diverse computational workflows for scientific problem generation.

3.2 Operator Graph Sampling

Each node in the operator graph represents an API $\times$ mode combination, as defined in Section 3.1. Within each subdomain, the framework scores directed connections between operators according to their input/output compatibility and operational workflows, retaining multiple high-scoring successors for each node. Operator chains are then sampled through random walks. Starting from a selected operator, the framework repeatedly chooses among its retained successors until the target chain length is reached. Varying the starting node and walk path produces diverse operator combinations from the same graph. The resulting chains and their node descriptions serve as computational workflow cues for subsequent scientific problem design.

3.3 Scientific Problem Generation

Given a sampled operator chain, the framework generates a problem through three stages: (1) **Chain assessment**, it evaluates whether the chain can support a coherent computational workflow with a natural scientific context, well-defined target quantities, and meaningful multistep reasoning. (2) **Context construction**, the framework specifies the input conditions, solution objectives, and roles of individual operators, followed by an independent plausibility review. Rejected contexts are regenerated and reviewed within a limited number of rounds. (3) **Problem authoring**, using the approved context and operator chain, the framework generates the scientific task and substeps, tests, a reference implementation based on basic numerical functions, and an oracle implementation using high-level scientific APIs. The problem statement is then revised for consistency with the generated code and tests.

The final problem contains the main problem, subproblems, input/output specifications, reference solutions, and tests. Substeps are organized according to the scientific task and its dependencies, after which the problem proceeds to execution validation and repair.

3.4 Validation, Repair, and Final Refinement

After a problem is generated, it undergoes three stages to ensure its executability and overall quality: (1) **Execution validation**, the framework checks the problem structure, interfaces, and dependencies, executes substep and end-to-end tests, and performs differential testing between the reference and oracle implementations. The tests cover numerical correctness, parameter variations, boundary cases, and scientific invariants. (2) **Execution-guided repair**, when validation fails, error messages are

fed back to the model to repair the problem statement, tests, or implementations. Each revision is revalidated, and candidates that exceed the repair limit are discarded. (3) **Quality refinement**, validated problems are polished and assessed for scientific naturalness, task difficulty, implementation complexity, and test strength. Candidates below the quality threshold undergo limited rounds of refinement and renewed validation, after which only qualified problems are retained.

3.5 SciWalker Implementation and Dataset Statistics

We use DeepSeek-V4-Flash (DeepSeek-AI, 2026) throughout the entire data generation pipeline. With stage-specific prompts, the same model performs chain review, context selection, problem generation, execution-guided repair, polishing, and quality review. We enable its maximum reasoning setting and use a context window of 200,000 tokens.

We organize APIs from scientific Python libraries (Appendix E) into subdomain-specific operator catalogs, which serve as the basis for our data generation pipeline. The resulting problems span five broad scientific domains—mathematics, physics, chemistry, biology, and materials science, covering 32 subdomains. Figure 3(a) shows the distribution of these subdomains across the five domains. For each subdomain, we construct 560 operators, yielding 17,920 operators and 772,178 graph edges in total. Operator chains of lengths 3-15 are then sampled as workflow cues for problem generation.

After execution-based validation and quality filtering, we retain 8,178 high-quality problems, whose domain distribution is shown in Figure 3(b). Appendix B reports candidate retention across the generation stages. Each problem comprises 3–6 substeps, resulting in 37,820 substeps overall, with a mean of 4.62 and a median of 5 per problem; the corresponding distribution is shown in Figure 3(c). The number of substeps does not directly correspond to operator-chain length. As discussed in Section 3.3, operator chains provide high-level computational workflow cues, while the model structures the final substeps according to the scientific objective and computational dependencies of each problem.

4 Reinforcement Learning for Scientific Coding

To evaluate the effectiveness of the data generated by SciWalker for training scientific coding models, we perform reinforcement learning with execution-based rewards. We use GSPO for policy optimization with substep execution rewards.

4.1 Policy Optimization with GSPO

We optimize the policy with Group Sequence Policy Optimization (GSPO), using the GSPO-token formulation introduced by Zheng et al. (2025). For each main problem, we sample multiple complete solution trajectories and expand them into substep-level prompt–response samples. Each prompt contains the current task and preceding code from the same trajectory. The response contains the current substep’s reasoning and final code. Only the generated response tokens contribute to the loss. Responses to the same substep of the same main problem form a group, although their preceding code may differ. Within each group, the advantage of trajectory i at substep t is

$$A_{i,t} = \frac{r_{i,t} - \mu_t}{\sigma_t + \delta}, \quad (1)$$

where $r_{i,t}$ is the training reward defined in Equation 5, μ_t and σ_t are the group reward mean and sample standard deviation, and δ is a small constant for numerical stability.

For policy optimization, we denote a substep sample by (x_i, y_i, A_i) , where y_i is the complete response for one substep. GSPO uses the length-normalized sequence probability ratio

$$\rho_i(\theta) = \left(\frac{\pi_\theta(y_i | x_i)}{\pi_{\text{old}}(y_i | x_i)} \right)^{1/|y_i|}, \quad (2)$$

where π_{old} is the fixed old policy. GSPO-token uses

$$\hat{\rho}_{i,k}(\theta) = \text{sg}[\rho_i(\theta)] \frac{\pi_\theta(y_{i,k} | x_i, y_{i,<k})}{\text{sg}[\pi_\theta(y_{i,k} | x_i, y_{i,<k})]}, \quad (3)$$

where k indexes response tokens and `sg` stops gradient propagation. Every token shares the sequence ratio’s forward value, while gradients propagate through its own log probability. The GSPO-token objective is

$$\mathcal{J}_{\text{GSPO-token}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{1}{|y_i|} \sum_{k=1}^{|y_i|} \min[\hat{\rho}_{i,k}(\theta) A_i, \text{clip}(\hat{\rho}_{i,k}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_i], \quad (4)$$

where $\mathcal{B}$ denotes a batch of substep training samples, and ϵ_{low} and ϵ_{high} specify the clipping range.

All tokens in a substep response share the same advantage A_i , so this formulation is equivalent to GSPO in objective value, clipping conditions, and gradient. Additional rollout probability correction and numerical safeguards used in training are detailed in Appendix D.

4.2 Constructing Substep Training Samples

After complete trajectories are generated and scored, they are expanded into training samples by substep. Let main problem q have T_q steps requiring model responses. Each trajectory then yields T_q prompt-response samples, and eight trajectories yield $8T_q$ samples in total. The sample for trajectory i at step t is denoted by $(x_{q,i,t}, y_{q,i,t})$, where $x_{q,i,t}$ contains the current task, interface requirements, and preceding code from the same trajectory, while $y_{q,i,t}$ contains the reasoning and final code generated for the current step.

Each sample is optimized only over the valid generated tokens in its current response, with both reasoning and the final code contributing to the loss. Preceding code belongs to the prompt and is not counted again in the loss as a prediction target for the current step. Each substep retains its own reward, and rewards from subsequent steps are not accumulated backward.

Each substep receives an execution reward of 1 if all its tests pass and 0 otherwise:

$$r_{i,t} = \mathbf{1}[\text{all tests for substep } t \text{ pass}]. \quad (5)$$

5 Experiments

5.1 RL Training and Evaluation Settings

The shared training and evaluation settings are summarized in Table 1. As an additional baseline, we apply PPO-based RL with a 50-step warm-up schedule.

Evaluation. We adopt 16 different evaluation benchmarks to evaluate the training effectiveness, which comprise five benchmarks for scientific computing and code generation, six for code repair and execution, and five for reasoning and knowledge:

1. Scientific computing and code generation: SciCode (Tian et al., 2024), DS-1000 (Lai et al., 2023), HumanEval (Chen et al., 2021), LiveCodeBench Code Generation v6 (Jain et al., 2025), and APPS Introductory (Hendrycks et al., 2021);
2. Code repair and execution: HumanEvalFix-Python, HumanEvalFix-Rust, and HumanEvalFix-JavaScript (Muennighoff et al., 2024), QuixBugs-Java and QuixBugs-Python (Lin et al., 2017), and LiveCodeBench Execution v2 (Jain et al., 2025);
3. Reasoning and knowledge: MATH-500 (Lightman et al., 2024), GSM8K (Cobbe et al., 2021), BBH Multistep Arithmetic (Suzgun et al., 2023), ARC-Easy (Clark et al., 2018), and MMLU-Pro Computer Science (Wang et al., 2024).

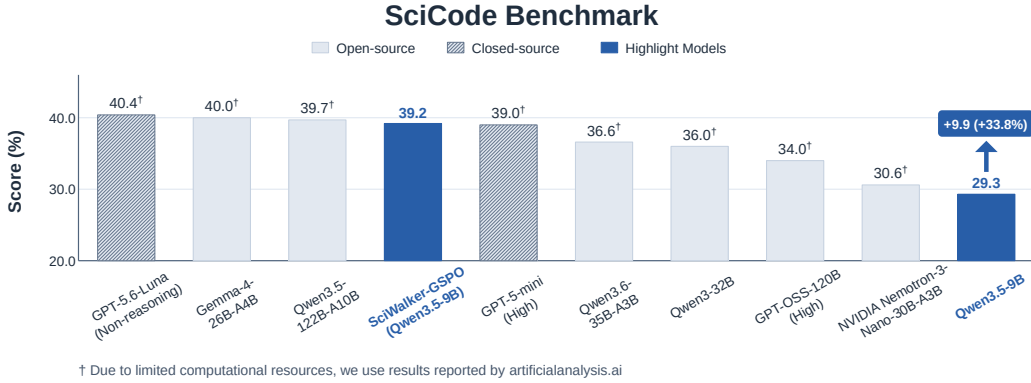
Each trained model uses a fixed checkpoint across benchmarks, and the best checkpoint is selected via the evaluation score of SciCode. Across the 16 benchmarks, we calculate the unweighted mean of the benchmark scores, each averaged over three evaluations.

5.2 Results Comparison

SciCode Performance. Figure 4 reports model performance on SciCode, combining our local evaluations with externally reported results. Each local evaluation covers 288 test-set subproblems

Table 1: Shared reinforcement learning and evaluation settings.

Item	Shared setting
Learning rate	1×10^{-6}
Optimizer parameters	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay = 0.1
GSPO clipping	$\epsilon_{\text{low}} = 3 \times 10^{-4}$, $\epsilon_{\text{high}} = 4 \times 10^{-4}$
Gradient clipping	1.0
KL / entropy terms	KL reward, KL loss, and entropy regularization disabled
Training budget	150 steps, 32 problems per step, 8 trajectories per problem
Training sampling	temperature=1.0, top_p=1.0, top_k=-1
Training input / generation limit	40,000 / 20,000 tokens
Training subproblem execution timeout	120 seconds
Evaluation settings	Thinking, generation limit of 32,768 tokens, execution timeout of 300 seconds
Evaluation sampling	temperature=0.6, top_p=0.95, top_k=20


Figure 4: SciCode performance comparison. Local evaluations report mean subproblem accuracies over three evaluations of 288 subproblems with scientific background. External model scores are reported by Artificial Analysis (Artificial Analysis, 2026).

with scientific background. By applying GSPO with SciWalker-generated training data, Qwen3.5-9B improves from 29.3% to **39.2%**, **an absolute gain of 9.9 points** (33.8% relative). It surpasses substantially larger models such as Qwen3-32B (36.0%) and GPT-OSS-120B (34.0%), while matching GPT-5 mini (39.0%) and approaching Qwen3.5-122B (39.7%). These results demonstrate the effectiveness of SciWalker-generated data in substantially improving scientific reasoning capabilities, particularly for compact models.

Cross-Benchmark Performance. Table 2 compares the Qwen3.5-9B baseline with PPO and GSPO across 16 benchmarks. Overall, GSPO achieves substantial improvements over the baseline across a broad range of benchmarks. The gain is particularly pronounced on SciCode, where GSPO improves subproblem accuracy by 9.9 percentage points, demonstrating the effectiveness of the scientific-coding data generated by SciWalker. This benefit generalizes to other coding tasks that GSPO improves DS-1000 by 17.3 points, LiveCodeBench Code Generation v6 by 13.0 points, and APPS Introductory by 15.5 points.

The improvements also extend to out-of-domain tasks that are not directly targeted by the SciWalker data. On code repair, GSPO improves HumanEvalFix-Python and HumanEvalFix-JavaScript by 13.4 and 47.8 points, respectively. It also gains 6.7 points on QuixBugs-Java and 10.0 points on QuixBugs-Python. Moreover, GSPO consistently improves all five reasoning and knowledge benchmarks, such as gains of 6.0 points on MATH-500 and 4.9 points on GSM8K. These improvements are notable because these benchmarks are not coding tasks, suggesting that learning from high-quality scientific-coding trajectories can strengthen more general reasoning capabilities. Overall, GSPO

Table 2: Performance of the Qwen3.5-9B baseline, PPO, and GSPO across 16 benchmarks. Scores (%) are means over three evaluations, rounded to one decimal before calculating the parenthesized percentage-point changes relative to the baseline. Increases and decreases are shown in dark blue and dark red, respectively. SciCode reports subproblem accuracy, code benchmarks report pass@1, and reasoning and knowledge benchmarks report accuracy. Bold indicates the highest mean score in each column, including ties.

Scientific computing and code generation						
Model	SciCode	DS-1000	HumanEval	LCB Code Gen. v6	APPS Introductory	
Baseline	29.3	45.6	94.7	61.9	69.1	
PPO	32.6 (+3.3)	49.4 (+3.8)	93.5 (-1.2)	57.6 (-4.3)	72.3 (+3.2)	
GSPO	39.2 (+9.9)	62.9 (+17.3)	97.8 (+3.1)	74.9 (+13.0)	84.6 (+15.5)	
Code repair and execution						
Model	HumanEvalFix Python	HumanEvalFix Rust	HumanEvalFix JavaScript	QuixBugs Java	QuixBugs Python	LCB Execution v2
Baseline	80.5	51.2	43.3	58.3	77.5	97.1
PPO	82.7 (+2.2)	60.6 (+9.4)	66.9 (+23.6)	58.3 (0.0)	63.3 (-14.2)	97.8 (+0.7)
GSPO	93.9 (+13.4)	48.4 (-2.8)	91.1 (+47.8)	65.0 (+6.7)	87.5 (+10.0)	97.4 (+0.3)
Reasoning and knowledge						
Model	MATH-500	GSM8K	BBH Arithmetic	ARC-Easy	MMLU-Pro CS	
Baseline	86.1	91.1	92.9	98.9	84.3	
PPO	81.4 (-4.7)	90.4 (-0.7)	89.5 (-3.4)	98.8 (-0.1)	85.0 (+0.7)	
GSPO	92.1 (+6.0)	96.0 (+4.9)	97.1 (+4.2)	99.0 (+0.1)	84.6 (+0.3)	

* LCB denotes LiveCodeBench. BBH Arithmetic denotes BBH Multistep Arithmetic. HumanEvalFix uses the test version. LiveCodeBench Execution contains 479 execution instances from 92 source problems.

improves 15 of the 16 benchmarks and raises the average score from 72.6% to 82.0%, indicating both the quality of the SciWalker-generated data and its broad generalization value.

Although PPO is less effective and less consistent than GSPO, it still improves SciCode by 3.3 points and produces clear gains on several coding-related benchmarks, such as DS-1000 (+3.8), APPS Introductory (+3.2). These gains provide additional evidence that the SciWalker-generated data contain useful learning signals. However, PPO improves only eight benchmarks, while degrading seven, and increases the overall mean by only 1.2 points. Its regressions on LiveCodeBench Code Generation and most reasoning benchmarks suggest that PPO does not exploit these signals as reliably as GSPO. One possible explanation is that critic-based credit assignment introduces estimation errors that make optimization less stable and limit generalization, highlighting the importance of the training objective in fully realizing the value of the generated data.

6 Conclusion

In this study, we introduced SciWalker, a framework for synthesizing scientific coding problems through operator-chain sampling and execution feedback. By using sampled operator chains as computational workflow cues, SciWalker generates scientifically grounded and diverse problems and iteratively repairs failed generations. Using this framework, we constructed 8,178 high-quality problems spanning 5 scientific domains and 32 subdomains. We conducted reinforcement learning on Qwen3.5-9B using GSPO algorithm and the training improved SciCode subproblem accuracy by 9.9 percentage points, from 29.3% to 39.2%, showing the effectiveness of the generated coding data. We also show that human-authored open-source code, such as Python libraries, is a valuable resource for synthetic coding data generation.

Acknowledgments

This work was supported by the Shanghai Artificial Intelligence Laboratory. We are grateful to the authors and open-source communities whose work made this project possible.

References

- Artificial Analysis. SciCode benchmark leaderboard, 2026. URL <https://artificialanalysis.ai/evaluations/scicode>. Accessed September 23, 2026.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring Coding Challenge Competence With APPS. In J. Vanschoren and S. Yeung (eds.), *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/c24cd76e1ce41366a4bbe8a49b02a028-Paper-round2.pdf.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 58791–58831, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/94074dd5a072d28ff75a76dabed43767-Paper-Conference.pdf.
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 18319–18345. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/lai23b.html>.
- Baixuan Li, Jialong Wu, Yida Zhao, Wendong Xu, Xuanzhong Chen, Huifeng Yin, Liang Chen, Wentao Zhang, and Kuan Li. UniScientist: Advancing Universal Scientific Research Intelligence. UniPat AI technical blog, March 2026. URL <https://unipat.ai/blog/UniScientist>. March 4, 2026.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 39578–39601, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/aca97732e30bcf1303bc22ac3924fd16-Paper-Conference.pdf.
- Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. QuixBugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, SPLASH ’17*, pp. 55–56. ACM, October 2017. doi: 10.1145/3135932.3135941. URL <http://dx.doi.org/10.1145/3135932.3135941>.
- Tengxiao Liu, Deepak Nathani, Zekun Li, Kevin Yang, and William Yang Wang. WildSci: Advancing scientific reasoning from in-the-wild literature. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 11677–11695, San Diego,

- California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.567. URL <https://aclanthology.org/2026.findings-acl.567/>.
- Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. OctoPack: Instruction Tuning Code Large Language Models. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 7604–7623, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/1ec299a5229034141e58aeded0d0b9de-Paper-Conference.pdf.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.824. URL <https://aclanthology.org/2023.findings-acl.824/>.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, Xinan Chen, Cunwei Fan, Xuefei Guo, Roland Haas, Pan Ji, Kittithat Krongchon, Yao Li, Shengyan Liu, Di Luo, Yutao Ma, Hao Tong, Kha Trinh, Chenyu Tian, Zihan Wang, Bohao Wu, Yanyu Xiong, Shengzhu Yin, Minhui Zhu, Kilian Lieret, Yanxin Lu, Genglin Liu, Yufeng Du, Tianhua Tao, Ofir Press, Jamie Callan, Eliu Huerta, and Hao Peng. SciCode: A Research Coding Benchmark Curated by Scientists. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 30624–30650. Curran Associates, Inc., 2024. doi: 10.52202/079017-0963. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/36850592258c8c41cecdad3dea5ff7de-Paper-Datasets_and_Benchmarks_Track.pdf.
- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhira Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 95266–95290. Curran Associates, Inc., 2024. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad236edc564f3e3156e1b2feafb99a24-Paper-Datasets_and_Benchmarks_Track.pdf.
- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with OSS-instruct. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 52632–52657. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/wei24h.html>.
- Dan Zhang, Ziniu Hu, Sining Zhou, Zhengxiao Du, Kaiyu Yang, Zihan Wang, Yisong Yue, Yuxiao Dong, and Jie Tang. SciInstruct: a Self-Reflective Instruction Annotated Dataset for Training Scientific Language Models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 1443–1473. Curran Associates, Inc., 2024. doi: 10.52202/079017-0046. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/02ee6b7295f720407b56c457b34c54d5-Paper-Datasets_and_Benchmarks_Track.pdf.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group Sequence Policy Optimization, 2025. URL <https://arxiv.org/abs/2507.18071>.
- Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. OpenCodeInterpreter: Integrating code generation with execution and refinement. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 12834–12859, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.762. URL <https://aclanthology.org/2024.findings-acl.762/>.

A From an `fsolve` Operator Chain to a Scientific Training Problem

Using the actual problem `nonlinear_parameter_estimation`, this appendix illustrates how SciWalker converts an operator chain into a training problem.

A.1 Operators and Chain Sampling

This example comes from the category of root finding and nonlinear equation solving, which is associated with SciPy interfaces such as `root`, `fsolve`, `brentq`, and `newton`. Among these, `scipy.optimize.fsolve` is combined with 35 operation modes to form 35 operators.

The sampled chain contains 11 operators and 10 edges, covering operations such as example adaptation, sensitivity analysis, data filtering, solving, residual computation, and stability checking.

For example,

```
fsolve x single_case_execute -> fsolve x compute_residual
```

This connection provides the computational cue check residuals after solving.

A.2 From Scientific Context to a Concrete Problem

We use DeepSeek-V4-Flash. The operator chain receives a problem-generation potential assessment score of 7, and its scientific context is accepted after the first round of selection and review. The framework then generates the main problem, five substeps, student, oracle, and tests in a single complete problem-generation call.

The final problem concerns concentration decay in a chemical reaction. Given observations of time and concentration, the task is to fit the exponential model

$$y(t) = A \exp(-kt) + B \quad (6)$$

by estimating its parameters A , k , and B . The task requires first fitting the parameters, then filtering outlier observations using a residual threshold and refitting, returning the final parameters and the condition number of the approximate Hessian $J^T J$.

Table 3 summarizes the five substeps of this problem.

Table 3: Substeps of the generated nonlinear parameter estimation problem.

Substep	Computational task
1	Compute the residual vector and Jacobian matrix
2	Compute one Gauss–Newton parameter update
3	Solve for the parameters using Gauss–Newton iterations with backtracking
4	Compute the condition number of the approximate Hessian
5	Filter the data using an absolute residual threshold

A.3 Execution Validation and Feedback-Driven Repair

After initial generation, the framework runs problem checks and tests, then feeds the errors and the previous complete problem back for repair. This example first passes full validation after three rounds of repair (Table 4).

Four test snippets compare the reference and oracle implementations on the same inputs. Their number remains unchanged throughout repair, with one snippet modified during the first repair. This example illustrates the joint revision of the problem statement, implementations, and tests.

Table 4: Execution validation and feedback-driven repair of the example problem.

Version	Main changes and validation results during generation
Initial generation	The problem statement directly hints at interfaces; parameter recovery, differential testing, and other checks fail
First repair	Backtracking is added to student and oracle, and one differential test case is modified; parameter recovery errors remain after outlier handling
Second repair	A lower-bound constraint on the decay rate k is added to both implementations; full validation still fails
Third repair	The normal equations are solved using a least-squares method, and the iteration stopping criterion is adjusted; full validation passes

A.4 Quality Grading, Inclusion, and Revalidation

After the whole repair progress, the framework proceeds to problem statement polishing and quality grading. In this example, the polishing response is identical to the repaired content and passes revalidation. The problem ultimately passes model-based quality review with an overall score of 8 and is included as a high-quality problem. The entire generation process involves 9 large language model calls, covering potential assessment, context selection and review, initial problem generation, three rounds of repair, polishing, and quality review.

B Stage-Wise Retention Rates During Generation

We report the number of candidates retained and the retention rate at each stage of problem generation (Table 5) to quantify the extent of filtering. All retention rates below are calculated relative to the initial 66,889 candidate operator chains.

Table 5: Candidate retention during SciWalker data generation. All rates use the initial 66,889 candidate operator chains as the denominator.

Stage	Retained	Reduction	Cumulative retention
Candidate operator chains	66,889	—	100.0%
Problem-generation potential assessment	21,543	45,346	32.2%
Scientific context selection and review	21,527	16	32.2%
Training problem generation and execution validation (including repair)	17,166	4,361	25.7%
Problem statement polishing and quality grading (retaining only high-quality problems)	8,178	8,988	12.2%

The largest reduction occurs during problem-generation potential assessment, where 45,346 candidates are discarded. Because connecting operators into a chain in the graph does not guarantee that they can naturally form a task with a scientific rationale. After this stage, 21,543 candidate chains proceed to subsequent stages, accounting for 32.2% of the initial candidates.

During scientific context selection and review, the model designs a scientific setting for each candidate chain and reviews the suitability of the solution objective and combination of operations. 21,527 candidate chains proceed to training problem generation, a reduction of 16 from the previous stage. This result includes context reselection.

During training problem generation and execution validation, the model generates problem statements, reference implementations, and tests. The framework runs checks and feeds errors back to the model for repair. Ultimately, 17,166 problems pass execution validation, a reduction of 4,361 from the previous stage, accounting for 25.7% of the initial candidates.

During problem statement polishing and quality grading, the model first polishes the problem statements based on the code and tests, then reviews the problems for scientific naturalness, task difficulty, implementation complexity, and test strength. Only 8,178 high-quality problems are retained, accounting for 12.2% of the initial candidates.

C Generation Behavior of the Baseline Model

This appendix examines the generation behavior of the Qwen3.5-9B baseline.

C.1 Task and Evaluation Settings

SciCode 14.1: Problem Statement and Scientific Background.

Problem statement

Implement a python function to employ Mannella’s leapfrog method to solve the Langevin equation of a microsphere optically trapped in the gas with the given initial condition.

Scientific background

For a microsphere trapped in the gas, we have the following Langevin equation:

$$\frac{d^2x}{dt^2} + \frac{dx}{dt}/\tau_p + \omega_0^2 x = \sqrt{\frac{2}{\tau_p}} v_{rms} \zeta(t), \quad (7)$$

where ω_0 is the resonant frequency of the optical trap, τ_p is the momentum relaxation time of the particle, v_{rms} is the root mean square velocity of the particle and $\zeta(t)$ is a normalized white-noise process. This stochastic differential equation can be rewritten as:

$$v = \frac{dx}{dt} \quad (8)$$

$$\frac{dv}{dt} = -v/\tau_p - \omega_0^2 x + \sqrt{\frac{2}{\tau_p}} v_{rms} \zeta(t) \quad (9)$$

Mannella’s leapfrog method with step-size Δt defined as:

$$x_{n+1/2} = x_n + v_n \Delta t / 2, \quad (10)$$

$$v_{n+1} = (v_n - v_n \Delta t / (2\tau_p) - \omega_0^2 x_{n+1/2} \Delta t + \sqrt{\frac{2}{\tau_p}} v_{rms} \Delta W) / (1 + \Delta t / (2\tau_p)), \quad (11)$$

$$x_{n+1} = x_{n+1/2} + v_{n+1} \Delta t / 2, \quad (12)$$

where ΔW is sampled from a Gaussian distribution with mean zero and standard deviation $\sqrt{\Delta t}$.

Required dependency

```
import numpy as np
```

Function interface and input/output specification

```
def harmonic_mannella_leapfrog(x0, v0, t0, steps, taup, omega0, vrms):
    '''Function to employ Mannella's leapfrog method to solve the
    ↳ Langevin equation of a microsphere optically trapped in the
    ↳ gas.
    Input
    x0 : float
        Initial position of the microsphere.
    v0 : float
        Initial velocity of the microsphere.
    t0 : float
        Total simulation time.
    steps : int
```

```

        Number of integration steps.
    tau_p : float
        Momentum relaxation time of the trapped microsphere in the gas
        ↪ (often referred to as the particle relaxation time).
    omega_0 : float
        Resonant frequency of the harmonic potential (optical trap).
    v_rms : float
        Root mean square velocity of the trapped microsphere in the
        ↪ gas.
    Output
    x : float
        Final position of the microsphere after the simulation time.
    '''

```

The problem requires passing the updated position and velocity to the next integration iteration. The omitted velocity-state update examined below occurs at this point.

The baseline uses the Inspect evaluation protocol. Scientific background is provided, thinking is enabled, the generation limit is 32,768 tokens, temperature=0.6, top_p=0.95, top_k=20, and the subproblem execution timeout is 300 seconds. The baseline is evaluated independently three times.

C.2 Results and Generation Lengths Across Three Evaluations

On this subproblem, the baseline passes none of the three evaluations (Table 6).

Table 6: SciCode 14.1: baseline results and generation lengths across three independent evaluations. Length includes both reasoning and the final answer.

Evaluation	Tokens	Result
First	1,496	Generates code but omits the velocity-state update; fails the tests
Second	1,326	Generates code but omits the velocity-state update; fails the tests
Third	32,768	Repeatedly discusses dependency imports, reaches the length limit, and produces no final code
Mean / Total	11,863.33	0/3 pass

Generation length is the total number of generated tokens, including thinking text and the final answer. One baseline response is truncated. None of the three evaluations encounters an execution timeout.

C.3 Persistent Looping and State Updates

Dependency import loop. In the third evaluation, the baseline repeatedly alternates between import NumPy for execution and follow the evaluation prompt and do not redeclare dependencies, without making further progress on the integration implementation. After removing blank lines and leading and trailing whitespace, the longest consecutive sequence of complete repetitions of the same four-line unit spans 308 iterations. This continuous region accounts for 82.11% of the thinking text (by character count). Generation eventually exhausts the 32,768-token budget and terminates with `max_tokens`, leaving the final code empty.

Velocity-state update. In the first and second evaluations, the baseline generates code but, after computing `v_new`, uses it only to update the position without updating the velocity variable `v` to `v_new`. The next iteration therefore continues to read the old velocity, violating the stepwise update requirement in Section C.1.

D Rollout Probability Correction and Numerical Details

This appendix describes the additional rollout probability correction and numerical safeguards used with the GSPO-token formulation in Section 4.1. Let $m_{i,k}$ mark valid generated tokens. We denote current-policy token log probabilities by $\ell_{i,k}^\theta$, old-policy log probabilities recomputed on the training

side by $\ell_{i,k}^{\text{old}}$, and log probabilities recorded during rollout by $\ell_{i,k}^{\text{roll}}$. All probabilities are conditioned on the sample’s actual prompt and preceding response tokens.

The token-level rollout correction weight is

$$c_{i,k} = \text{sg}[\min(2, \exp[\text{clip}(\ell_{i,k}^{\text{old}} - \ell_{i,k}^{\text{roll}}, -20, 20)])]. \quad (13)$$

These weights are computed and fixed before the policy update. We do not normalize them across the batch or reject trajectories based on these weights. They multiply the token losses without modifying rewards or group-relative advantages.

For numerical stability, Equation 3 is evaluated in log space with its exponent capped at 10. Writing $\bar{\Delta}_i = \log \rho_i(\theta)$, the resulting ratio is

$$\tilde{\rho}_{i,k} = \exp[\min(\text{sg}(\bar{\Delta}_i) + \ell_{i,k}^\theta - \text{sg}(\ell_{i,k}^\theta), 10)]. \quad (14)$$

With $\epsilon_{\text{low}} = 3 \times 10^{-4}$ and $\epsilon_{\text{high}} = 4 \times 10^{-4}$, the implemented loss combines this ratio with the rollout correction weights:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{\sum_k m_{i,k} c_{i,k} \min[\tilde{\rho}_{i,k} A_i, \text{clip}(\tilde{\rho}_{i,k}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_i]}{\sum_k m_{i,k} + 10^{-8}}. \quad (15)$$

The loss is averaged over valid tokens within each response and then over substep samples. Both reasoning and final code are included. Prompt and padding tokens are excluded. We use $\delta = 10^{-6}$ in Equation 1, with no Critic, KL loss, or entropy regularization.

E Scientific Python Libraries

The operator catalogs draw on APIs from 79 Python libraries and package namespace groups across five scientific domains. Table 7 lists these libraries by domain, with shared libraries appearing in multiple rows. Their APIs provide computational workflow cues for problem generation.

Table 7: Scientific libraries and package namespace groups used for operator construction, grouped by domain. Library names are alphabetized within each row.

Domain	Libraries
Mathematics	arch, ArviZ, CVXPY, geomdl, JAXopt, NLOpt, NumPy, PyGMO, PyLops, PyMC, pymoo, Pyomo, PyVista, Riskfolio-Lib, scikit-learn, SciPy, SfePy, statsmodels, trimesh
Physics	alchemlyb, ASE, Astropy, Awkward Array, boost-histogram, Cirq, coffea, DecayLanguage, Diffractio, discretize, FiPy, Gala, galpy, GWpy, HCIPy, healpy, hist, Lightcurve, mplhep, NumPy, particle, POPPY, py_pol, PyDMD, PySINDy, Qiskit, QuTiP, qutip-qip, REBOUND, SciPy, sisl, Stim, SymPy, zfit
Chemistry	Biopython, Cantera, datamol, DeepChem, edlib, molmass, MolVS, OpenMM, parasail, periodictable, pysam, PySCF, RDKit, scikit-bio, SciPy
Biology	agentpy, Biopython, COBRApy, DendroPy, edlib, GillesPy2, Mesa, msprime, NumPy, parasail, PyMC, pysam, scikit-bio, SciPy, tskit
Materials science	CHGNet, Dans_Diffraction, datamol, jarvis-tools, jobflow, matminer, molmass, periodictable, pyFAI, pymatgen, RDKit